

Proyecto DBSCAN paralelo para la detección de ruido o outliers con OpenMP

Cómputo Paralelo y en la Nube
Fecha: 20 de octubre de 2026

Integrantes:
• José Malfavaun Gorostizaga
• Natalia Gómez Álvarez

Contenido

Resumen 3
Objetivo y definición del experimento 3
Algoritmo y alternativas de paralelización 3
Variables, configuraciones y repeticiones 4
Procedimiento y métricas 4
Hardware y software empleados 4
Integridad de los datos 5
Resultados Linux 5
Resultados Windows 9
Comparación entre los dos entornos 12
¿Cuál es mejor P1 o P2? 13
Uso responsable y ético de IA 13
Conclusiones 14

Resumen

Se evaluó una variante ingenua de dos etapas del algoritmo DBSCAN orientada a clasificar puntos bidimensionales como núcleo de primer orden, núcleo de segundo orden o ruido. Se comparó una implementación secuencial (serial) y dos estrategias paralelas con OpenMP: P1, que distribuye directamente las iteraciones por punto, y P2, que divide los puntos en k bloques y reúne los resultados mediante operaciones de merge. La base de datos original registra 2,320 ejecuciones, agrupadas en 232 configuraciones con 10 repeticiones cada una. El speedup promedio de todas las configuraciones paralelas fue 1.796× para P1 y 1.939× para P2; la aceleración individual máxima fue 5.749× en P2 con 200,000 puntos, 4 bloques y 8 hilos.

Estas medidas describen el conjunto evaluado del lado Linux y no implican que todas las configuraciones superen el umbral de 1.5×. En promedio, P2 ofrece una ligera ventaja sobre P1 con estos datos; P1 puede alcanzar resultados equivalentes en la mayoría de los tamaños salvo en n = 200,000, donde se queda justo en el umbral (1.553×).

Objetivo y definición del experimento

El objetivo fue determinar cuánto mejora el tiempo de cómputo al paralelizar la clasificación, cómo cambia esa mejora con el tamaño de entrada y el número de hilos, y si particionar explícitamente el trabajo en bloques ofrece ventajas frente al reparto directo de OpenMP. Para aislar el efecto de la paralelización se utilizaron los mismos archivos de puntos, epsilon=0.03 y min_samples=10 para las tres implementaciones.

Algoritmo y alternativas de paralelización

En el primer paso, cada punto se compara con los demás y se identifica como core de primer orden si encuentra al menos diez vecinos dentro de un radio epsilon de 0.03. En el segundo paso, los candidatos restantes se comparan con los puntos core: si alguno queda dentro del radio, se clasifica como core de segundo orden; en otro caso se considera ruido.

No se asignan identificadores de clúster. La implementación usa distancia euclidiana al cuadrado y corte anticipado cuando se alcanza el umbral; mantiene complejidad $O(n^2)$ en el peor caso. La versión serial ejecuta los dos pasos de forma secuencial.

Por otro lado, P1 utiliza #pragma omp parallel for schedule(dynamic, 64) para repartir los puntos entre hilos, con asignación dinámica que atenúa desbalances de carga entre puntos densos y aislados.

P2 forma k bloques contiguos y paraleliza el recorrido de esos bloques; cada bloque almacena resultados temporales en un buffer local, que después se combina con el resultado global. El merge del primer paso debe concluir antes de iniciar el segundo, porque un punto puede depender de un core identificado en otro bloque.

Variables, configuraciones y repeticiones

Variable	Valores y definición
Tamaño n	20'000, 40'000, 80'000, 120'000, 140'000, 160'000, 180'000, 200'000.
Número de hilos	1, 2, 4 y 8 en P1 y P2. En serial siempre 1
Bloques K de P2	2, 4, 8, 16, 32, 64
Parámetros del algoritmo	Epsilon = 0.03 y min_samples = 10 (mismos valores que usa DBSCAN_noise.ipynb)
Repeticiones	10 por configuración
Total	8x10 serial + (8x4x10) P1 + (8x6x4x10) P2 = 2'320 ejecuciones

Los datos de entrada se generaron con generar_puntos.py, que replica exactamente el método de la libreta oficial del curso (DBSCAN_noise.ipynb, repositorio del curso): make_blobs con 4 cúmulos gaussianos (centers=4, cluster_std=0.06) dentro de la caja [0,1]×[0,1], con valor absoluto y redondeo a 3 decimales, sin encabezado de texto en el CSV (formato idéntico al que produce np.savetxt en la libreta). El ruido que detecta DBSCAN surge naturalmente de la cola dispersa de cada cúmulo gaussiano, no se añade por separado.

Procedimiento y métricas

Se empleó el script run_reanudable.sh o run_windows.ps1 dependiendo del equipo que se utiliza, el cuál detecta configuraciones incompletas, ejecuta solo las repeticiones faltantes y agrega cada duración a la carpeta de datos como un archivo .csv con el nombre de resultados.

Los tiempos proceden de omp_get_wtime() antes del primer paso y después del segundo; no incluyen lectura ni escritura de CSV. Posteriormente, el programa llamado analizar.py, se encarga de calcular tanto el promedio como la desviación estándar de cada configuración y el speedup que se calcula de la siguiente manera: $\frac{\text{T_serial(n)}}{\text{T_paralelo(n,k,hilos)}}$

Para una comparación justa, la referencia serial se toma del mismo n. El análisis distingue rapidez absoluta (segundos), aceleración relativa (speedup), efecto de hilos, sensibilidad a k y variación entre repeticiones. Las mediciones son tiempos de pared y pueden verse afectadas por interferencias del entorno virtualizado.

Hardware y software empleados

Para la evaluación experimental se ejecutó el proyecto en dos diferentes entornos (Linux y Windows) para después poder realizar una comparación entre ambos resultados. Asimismo, se segura que en cualquiera de los dos se llega al speedup mínimo de 1.5x.

Características	Entorno Linux	Entorno Windows
Sistema Operativo	Ubuntu 22.04.5 LTS (Jammy Jellyfish), kernel 6.8, 64 bits	Windows 11 Pro, versión 10.0.262000, 64 bits
Procesador	ARM64 (aarch64) virtualizado — vCPU en contenedor de nube, sin modelo comercial expuesto	Intel Core i7-10510U @ 1.80GHz
Núcleos	4 vCPU (virtualizado; no aplica distinción física/lógica)	4 núcleos físicos y 8 procesadores lógicos
Memoria	3.8 GiB reportados	11.76 GiB reportados
Herramientas	GNU g++ 11.4.0, GNU Make 4.3, Python 3 (scikit-learn, numpy), OpenMP 4.5	MinGW g++, GNU Make 4.4.1, PowerShell 5.1, Python

Integridad de los datos

Se verificaron 2,320 filas válidas en el archivo de mediciones de Linux y de Windows, sin campos faltantes y con exactamente diez repeticiones para cada una de las 232 configuraciones. Los promedios remitidos concuerdan numéricamente con los recalculados a partir del CSV de mediciones. Los speedups se recalcularon desde los tiempos medios para evitar errores por redondeo o mezcla de conjuntos.

Resultados Linux

En el equipo de Linux la versión P1 obtuvo un speedup promedio de 1.796x con un rango de (0.697x, 2.833x); para P2 el speedup promedio fue de 1.939x con un rango de (0.575x, 5.749x). La media de P2 supera a la de P1 en aproximadamente 0.143×, una diferencia más marcada que con los parámetros anteriores. Con los cúmulos compactos de la libreta oficial (cluster_std=0.06), P2 saca más ventaja de dividir el trabajo en bloques que con la distribución anterior.

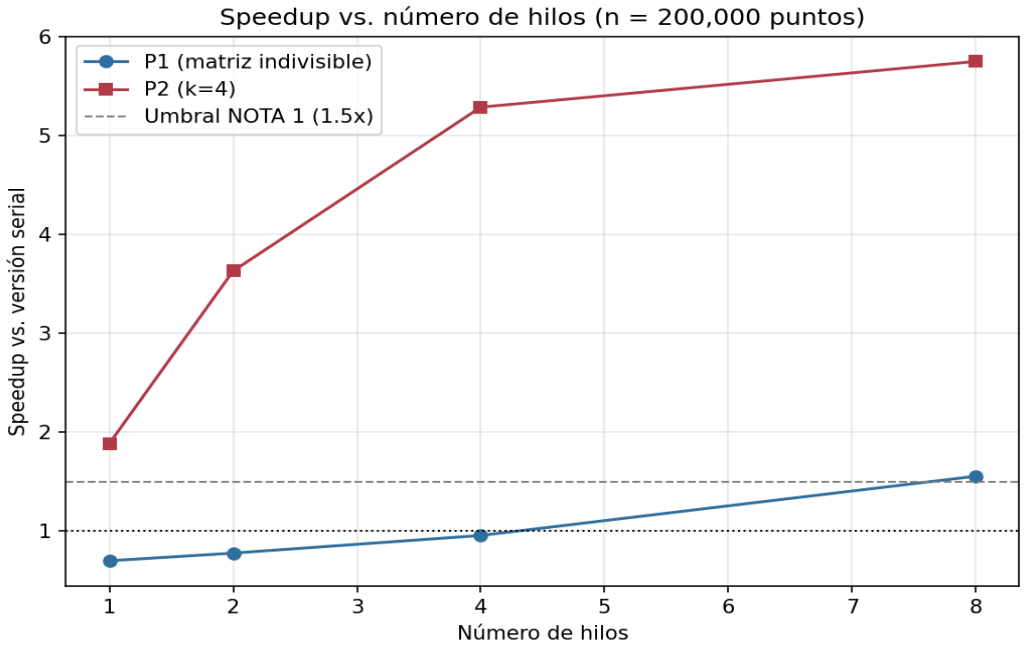


Figura 1. Linux — Speedup vs. número de hilos (n = 200,000 puntos).

Como se observa en la Figura 6, con un solo hilo ambas versiones paralelas muestran speedup por debajo de 1× (P1: 0.765x, P2: 0.890x), señal de sobrecosto de organización de la paralelización sin trabajo concurrente efectivo. De uno a cuatro hilos la aceleración aumenta claramente (P1: 2.431x, P2: 2.565x). De cuatro a ocho hilos P1 mejora poco (2.530x) mientras P2 se mantiene prácticamente igual (2.579x) en el promedio general — aunque, como se ve más abajo, en n = 200,000 específicamente P2 sí sigue mejorando mucho con 8 hilos.

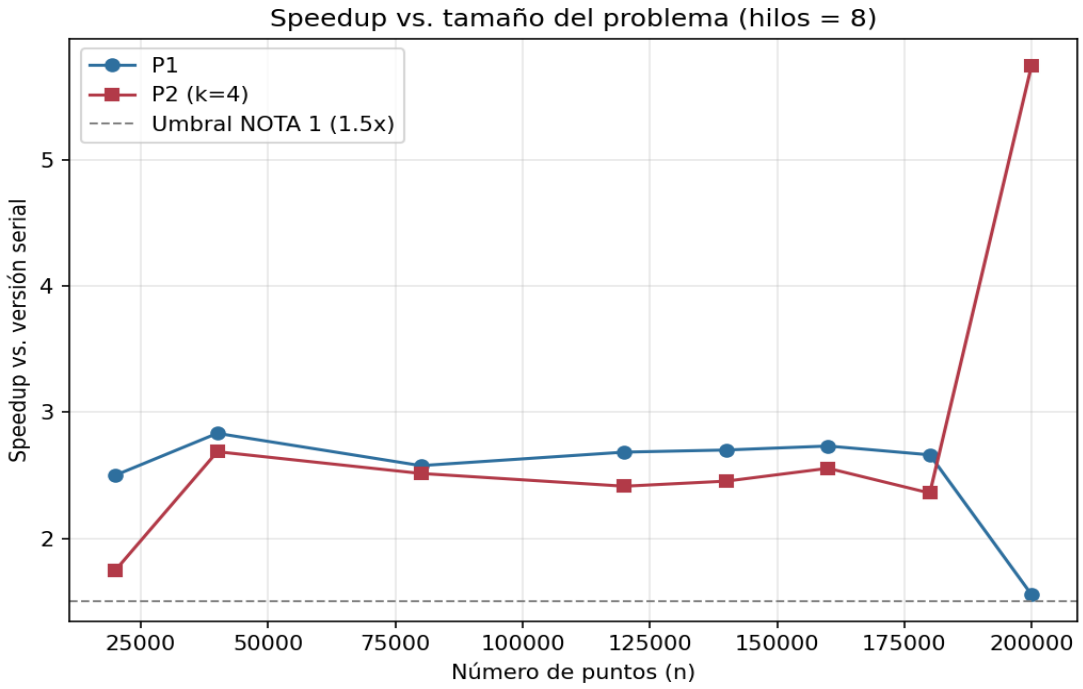


Figura 2. Linux — Speedup en función del tamaño n, con 8 hilos.

La Figura 7 muestra un patrón que vale la pena señalar: en n = 200,000 el speedup de P1 cae a su punto más bajo (1.553x, apenas sobre el umbral de 1.5x) mientras que el de P2 se dispara a su máximo (5.749x). Revisando los tiempos absolutos (Figura 9), el tiempo serial y el de P1 crecen de forma desproporcionada en ese tamaño —más de lo que indicaría una tendencia $O(n^2)$ suave— mientras P2 apenas cambia. Esto es consistente con que el costo del algoritmo depende de cuántas comparaciones se detienen anticipadamente tras encontrar los primeros min_samples vecinos: en una región donde eso ocurre con menos frecuencia, el trabajo real por punto sube más para la versión que no reparte en bloques pequeños.

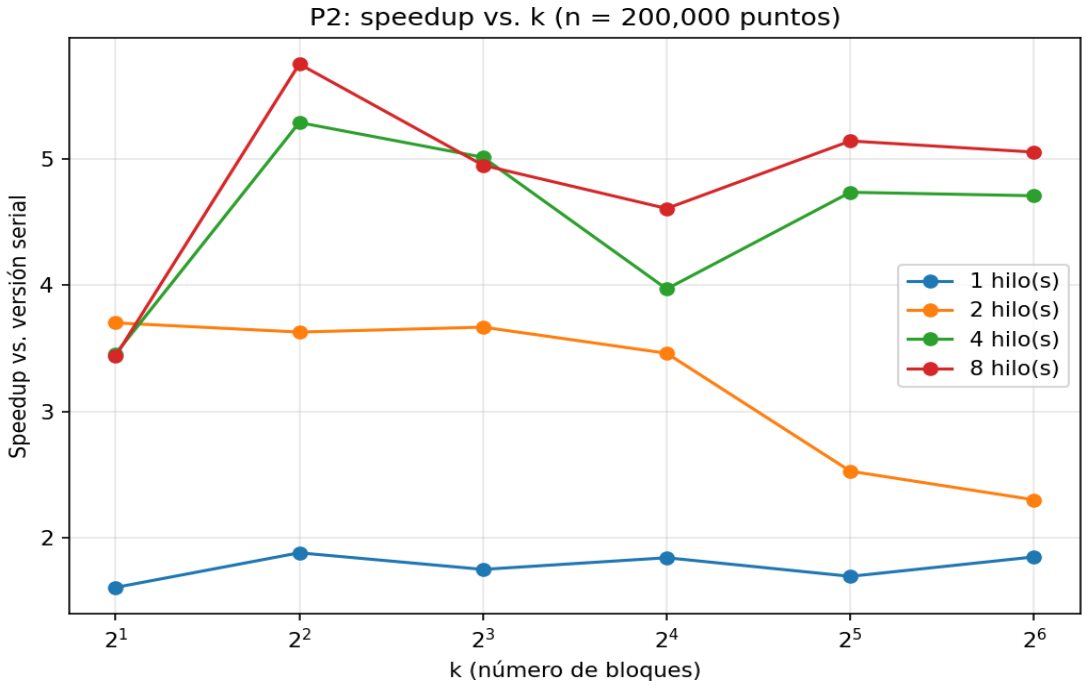


Figura 3. Linux — Dependencia del speedup de P2 respecto a k (n = 200,000 puntos).

En P2, el valor k controla el número de bloques, no el número de hilos. Para n = 200,000, el mejor desempeño con 8 hilos se obtiene en k = 4 (5.749x), seguido de k = 2 (3.44x aprox.) y valores más altos de k que oscilan entre 4.6x y 5.1x. A diferencia del experimento anterior (donde el óptimo estaba en k = 64), aquí los bloques grandes no ayudan tanto porque con cúmulos muy compactos el trabajo ya está relativamente balanceado con pocos bloques, y dividir de más solo añade costo de administración de buffers y merge.

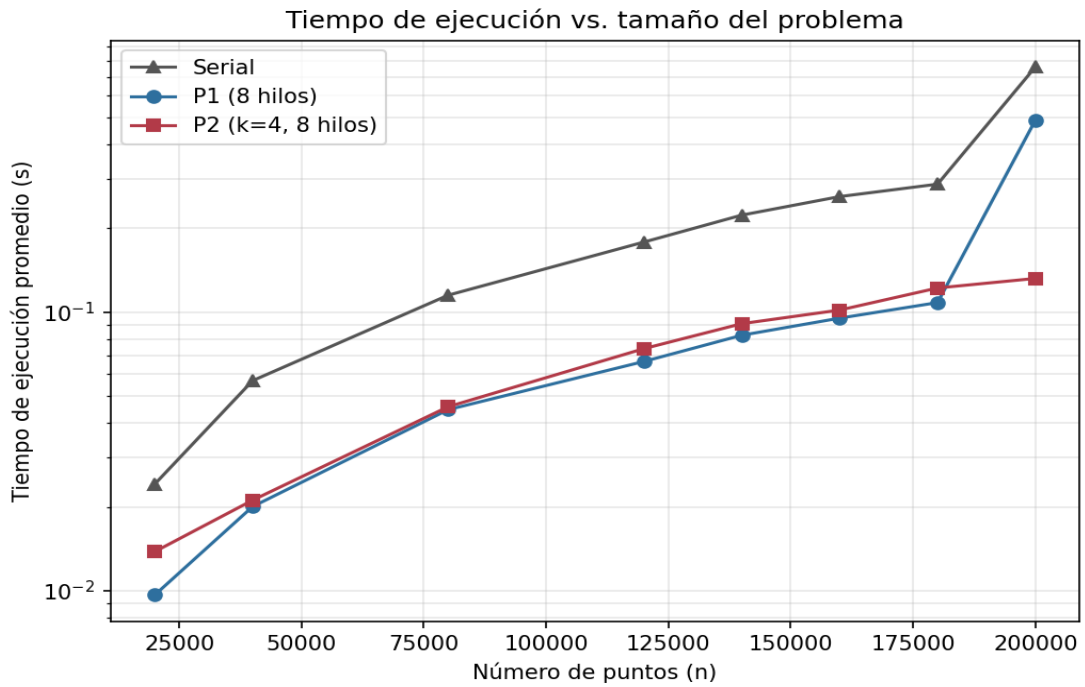


Figura 4. Linux — Tiempo de ejecución promedio frente al tamaño n (escala logarítmica, 8 hilos).

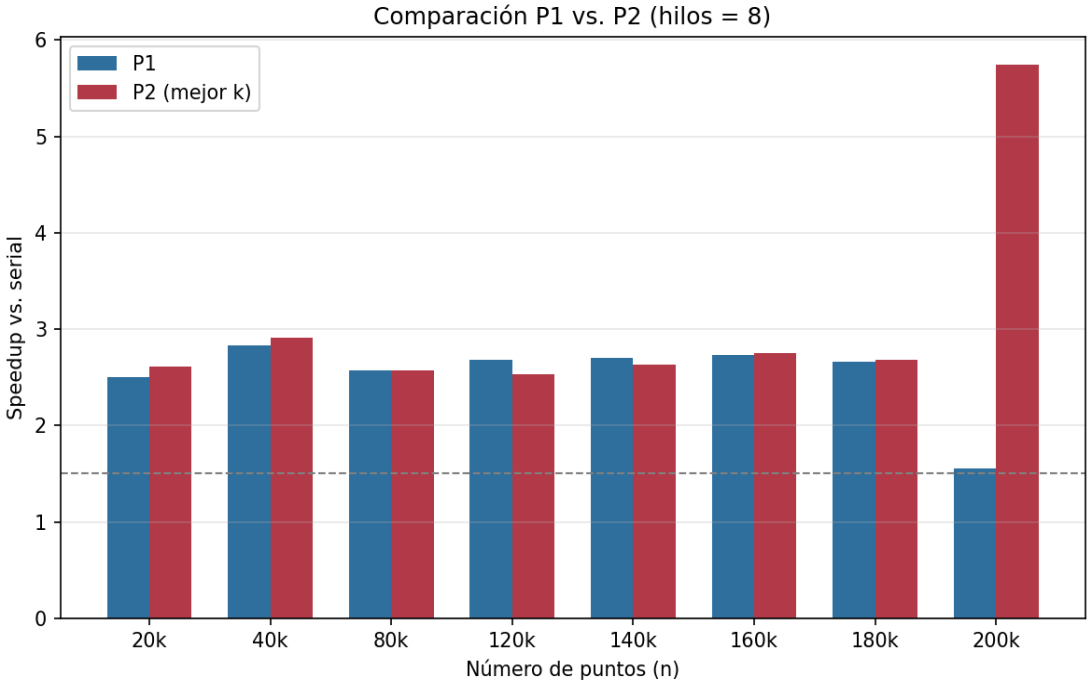


Figura 5. Linux — Comparación directa de P1 contra P2 (mejor k), 8 hilos.

El mejor registro individual (5.749x) ocurre en P2 para n = 200,000, k = 4 y 8 hilos. Entre las dos estrategias, con estos datos P2 obtuvo el mayor speedup promedio y el máximo puntual más alto; P1 se

mantuvo más estable en el rango medio de tamaños (20,000-180,000) pero fue la versión más afectada por la anomalía de n = 200,000.

Resultados Windows

La evaluación Windows se usaron los mismos archivos de puntos que Linux, así como epsilon = 0.03 y min_samples = 10. Las 2,320 mediciones se procesaron para calcular los resultados exactos: P1 promedió 2.208× y P2 1.999× en el conjunto de sus respectivas configuraciones; los máximos fueron 8.648× (P1, n=80,000, 8 hilos) y 6.942× (P2, n=80,000, k=8, 8 hilos).

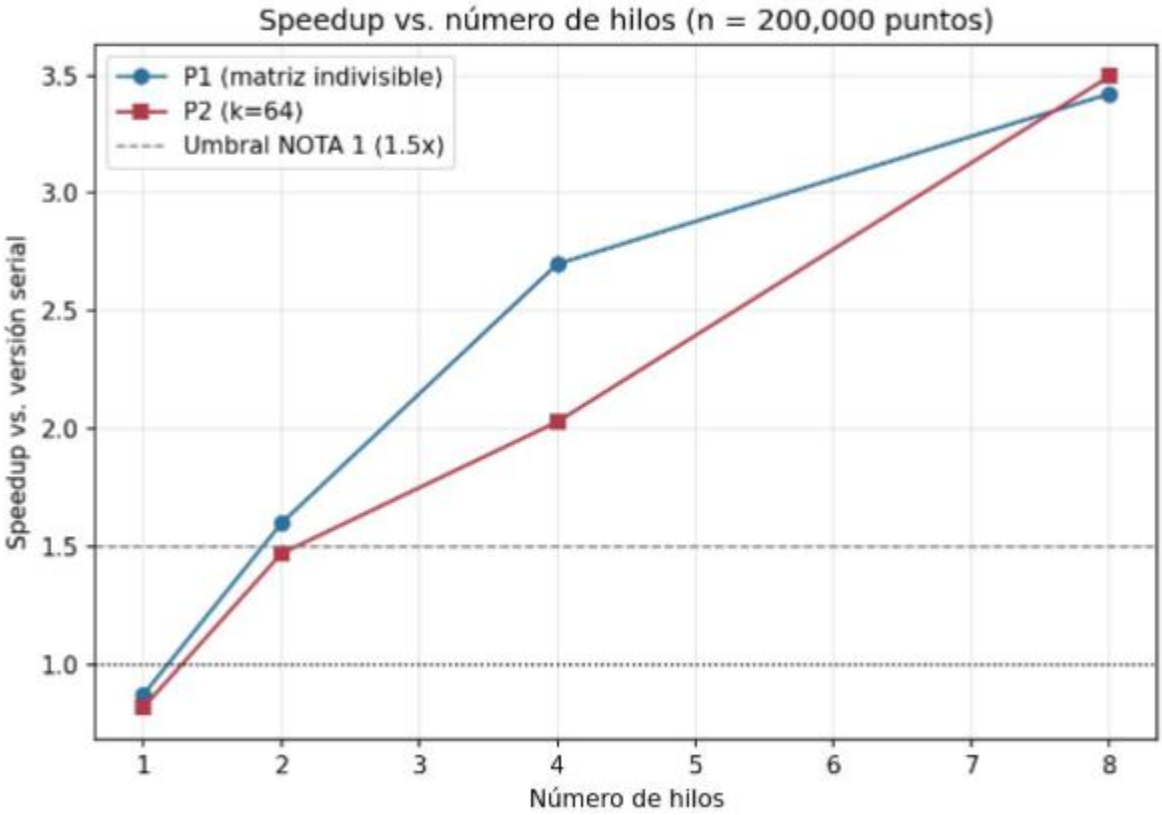


Figura 6. Windows — Speedup contra número de hilos para n = 200,000.

Con n = 200,000, ambas versiones mejoran al aumentar la cantidad de hilos. P1 pasa de una aceleración inferior a 1× con un hilo a aproximadamente 3.422× con ocho; P2 con k=64 alcanza aproximadamente 3.496× con ocho. Las configuraciones con un solo hilo incorporan sobrecosto de OpenMP —y, en P2, de buffers y merge— sin paralelismo real. Aunque el equipo tiene cuatro núcleos físicos, sus ocho procesadores lógicos permiten que ambas versiones continúen beneficiándose de ocho hilos en esta prueba particular.

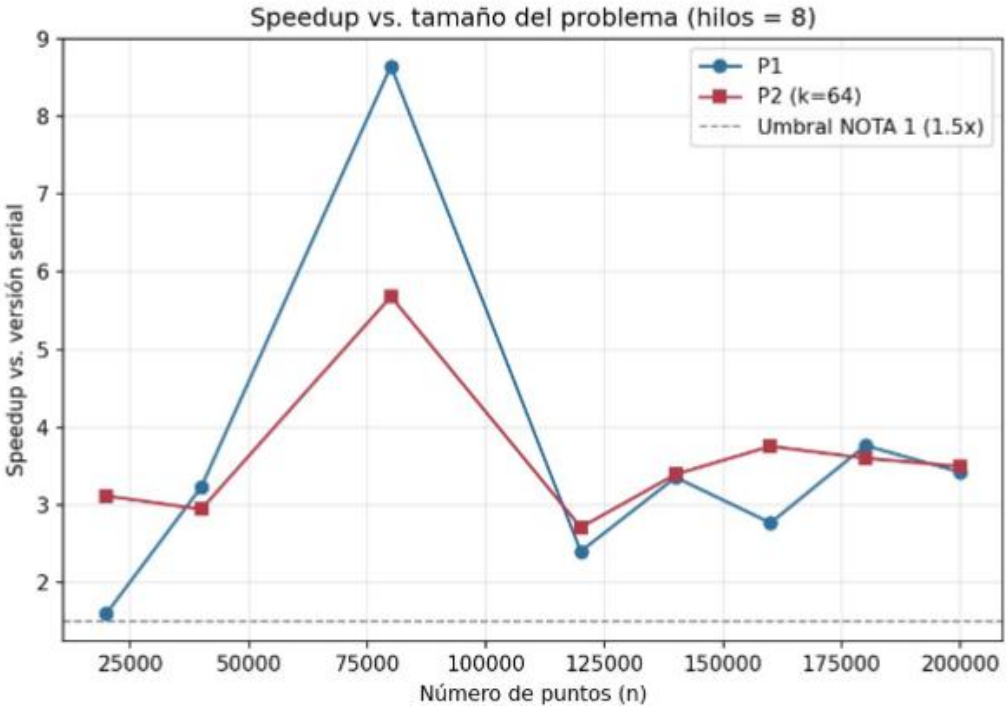


Figura 7. Windows — Speedup según el tamaño del problema, usando 8 hilos.

El speedup con ocho hilos depende del tamaño n y no crece de forma monotónica. P1 pasa de 1.600× para 20,000 puntos a un máximo excepcional de 8.648× con 80,000, mientras que P2 con k=64 llega allí a 5.682×. Para los 200,000 puntos, P1 obtiene 3.422× y P2 3.496×; las diferencias son pequeñas. El pico de 80,000 debe contextualizarse: el tiempo serial promedio fue 0.2534 s y su desviación estándar aproximadamente 0.0641 s, mientras que P1 promedió 0.0293 s, con desviación estándar cercana a 0.0013 s. La mayor variación de la referencia serial podría haber amplificado el cociente, por lo que 8.648× no debe afirmarse como escalabilidad superlineal sostenida.

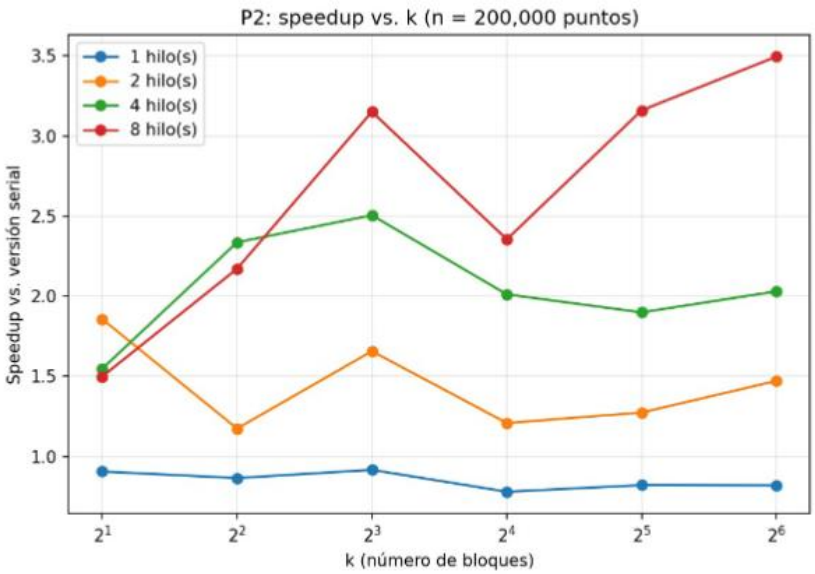


Figura 8. Windows — Efecto de k en P2 para n = 200,000

P2 es sensible al número k de bloques. Con n=200,000, el resultado de ocho hilos mejora desde aproximadamente 1.5× con k=2 hasta 3.496× con k=64, con fluctuaciones entre valores intermedios. Con cuatro hilos, la aceleración más alta se presenta aproximadamente cerca de k=8 y ronda 2.5×, mientras que con un único hilo no existe aceleración. Esto refleja el equilibrio entre disponer de tareas suficientes para balancear carga y el costo de gestionar particiones y realizar el merge; por ello k debe calibrarse según el tamaño y el hardware.

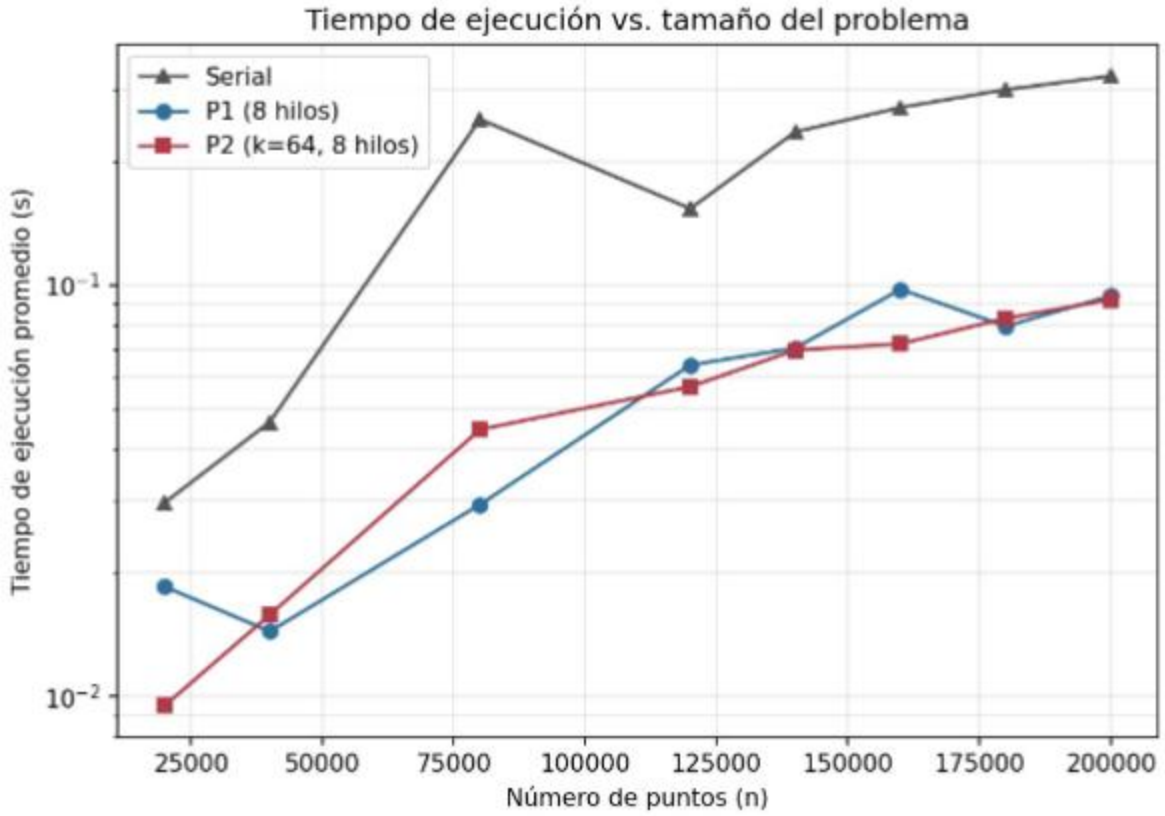


Figura 9. Windows — Tiempo promedio frente a n (escala logarítmica)

Los tiempos promedio de las implementaciones paralelas permanecen por debajo de los de la versión serial en los ocho tamaños de entrada mostrados. El eje vertical logarítmico permite apreciar su reducción relativa. Para n=80,000, el tiempo serial es anormalmente elevado frente a algunos tamaños mayores (promedio 0.2534 s), circunstancia coherente con el pico de speedup de la Figura 2. La forma no monótona de las curvas no contradice el límite asintótico $O(n^2)$: el número efectivo de comparaciones cambia debido al corte temprano al encontrar vecinos, a la distribución de los datos y a la variabilidad del equipo.

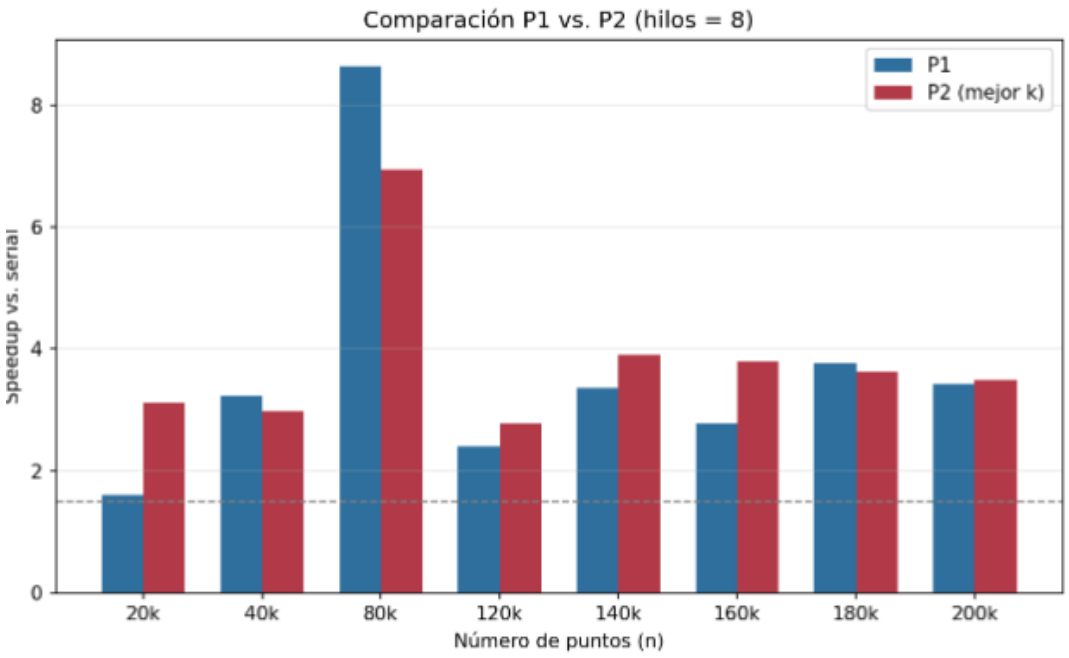


Figura 5. Windows — Comparación de P1 con P2 usando el mejor k para cada tamaño (8 hilos).

La gráfica enfrenta P1 con ocho hilos y P2 escogiendo el k con mayor aceleración para cada tamaño. P2 supera a P1 en cinco de los ocho tamaños (20,000; 120,000; 140,000; 160,000; 200,000), mientras P1 gana en los otros tres (40,000; 80,000; 180,000). La ventaja de P1 en n=80,000 es particularmente marcada: 8.648× frente a 6.942× del mejor P2. Debe precisarse que «P2 (mejor k)» es una selección a posteriori entre seis configuraciones, y por ello no equivale a ejecutar P2 con un único k predeterminado. A pesar de sus victorias locales, el promedio global de P2 (1.999×) fue inferior al de P1 (2.208×).

OBSERVACIÓN

El pico de P1 con 80,000 puntos es especialmente relevante. El tiempo serial promedio fue de 0.2534 segundos, mientras que P1 tardó 0.0293 segundos: $S_{P1} = \frac{0.2534}{0.0293} = 8.648$

Este valor supera ligeramente el número de procesadores lógicos utilizados (8 hilos), así que merece analizarse con cuidado. Puede estar relacionado con variaciones en las mediciones, efectos de memoria caché, frecuencia del procesador u otras condiciones de ejecución. No debemos atribuirlo automáticamente a una escalabilidad superior a la ideal.

Comparación entre los dos entornos

La evaluación utilizó la misma entrada en los dos entornos —se comprobó coincidencia de hash SHA256— y los mismos valores epsilon=0.03 y min_samples=10, con 1, 2, 4 y 8 hilos. En Linux (Ubuntu 22.04.5 virtualizado sobre ARM64 con 4 vCPU), la documentación reporta speedup promedio de 1.796× en P1 y 1.939× en P2. En Windows (Intel Core i7-10510U con 4 núcleos físicos y 8 procesadores lógicos) los promedios, recalculados del CSV, fueron 2.208× para P1 y 1.999× para P2. Así, Windows obtuvo

aproximadamente 0.412 puntos de speedup adicionales en P1 y 0.060 en P2, pero esas diferencias no deben atribuirse únicamente a Windows como sistema operativo: se comparan plataformas, virtualización y procesadores distintos.

La máxima aceleración reportada en Linux es 5.749× para P2 (n=200,000, k=4, ocho hilos). En Windows el máximo global es 8.648× para P1 (n=80,000, ocho hilos), seguido del máximo de P2 de 6.942× (n=80,000, k=8, ocho hilos). Estos picos corresponden a tamaños y particiones diferentes y, por sí solos, no establecen cuál computadora termina antes el mismo trabajo. Para n=200,000, la documentación Linux reporta 1.553× en P1 y 5.749× en P2 con k=4, mientras Windows obtiene 3.422× para P1 y 3.496× para P2 con k=64. La comparación de P2 en ese punto no corresponde al mismo k; debe leerse como comparación de configuraciones destacadas, no como experimento controlado de un parámetro idéntico.

En términos generales, las dos plataformas superaron la aceleración mínima de 1.5× requerida por el proyecto. Linux favoreció relativamente a P2, mientras Windows favoreció a P1 en el promedio de todas las configuraciones.

¿Cuál es mejor P1 o P2?

P1 distribuye directamente iteraciones individuales con #pragma omp parallel for schedule(dynamic,64). Su programación dinámica permite compensar la distinta carga de cada punto y evita buffers intermedios. P2 divide n en k segmentos, ejecuta cada segmento en un hilo disponible, conserva las clasificaciones en buffers locales y después realiza un merge explícito antes de formar la lista global de puntos core y comenzar el segundo paso. Ambos realizan esencialmente el mismo trabajo de clasificación sobre el conjunto completo de puntos.

Las implementaciones entregan las mismas clases de puntos, pero el modo de distribuir el trabajo difiere. P1 aplica schedule(dynamic,64) directamente sobre las iteraciones por punto: es simple, no requiere fijar un número de bloques y evita buffers y merge. P2 ofrece ajuste mediante k, pero agrega organización de tareas y combinación de resultados.

En Linux, P2 resultó superior en promedio (1.939× frente a 1.796× de P1) y alcanzó 5.749× en su mejor configuración. En Windows ocurrió lo contrario en el conjunto completo de configuraciones: P1 obtuvo 2.208× frente a 1.999× de P2 y el máximo más alto (8.648× frente a 6.942×). Sin embargo, P2 con el mejor k de cada tamaño ganó a P1 en cinco de ocho tamaños con ocho hilos. No hay, por tanto, una ganadora universal: **P1 se recomienda como elección general por simplicidad y buen promedio en Windows; P2 resulta atractiva cuando es posible ajustar k al tamaño y al entorno, como demuestran varias configuraciones y su ventaja media en Linux**

Uso responsable y ético de IA

Para este proyecto se utilizó una herramienta de inteligencia artificial generativa (Claude) como apoyo en la elaboración de una primera versión del código en C++ (serial, P1 y P2) y de los scripts de experimentación y análisis (generar_puntos.py, run_reanudable.sh, run_windows.ps1 y analizar.py). La herramienta no generó ni modificó datos, todas las mediciones provienen de ejecuciones reales de los programas en los equipos descritos.

El equipo revisó el código y lo compiló y ejecutó en ambos entornos. Verificó que las tres implementaciones clasifican igual los mismos puntos y que los parámetros (epsilon = 0.03, min_samples = 10, generación según DBSCAN_noise.ipynb) son idénticos. También recalculó promedios y speedups desde los CSV y contrastó cada gráfica con sus valores numéricos antes de interpretarla.

Conclusiones

El análisis experimental confirma la corrección funcional de la paralelización y el cumplimiento del requisito de aceleración superior a 1.5× en configuraciones de ambas versiones. En la máquina virtual Linux de 4 vCPU, P2 fue la estrategia con mejor speedup promedio (1.939×, frente a 1.796× de P1) y alcanzó 5.749×. En la computadora Windows, P1 encabezó la media global con 2.208× frente a 1.999× de P2, y logró el mayor valor puntual, 8.648× con 80,000 puntos. Ese máximo debe interpretarse con cautela por la dispersión del tiempo serial en ese tamaño.

La evidencia descarta una ventaja universal de P1 o P2: la elección depende de n, hilos, k y del equipo. P1 ofrece un mecanismo sencillo de reparto dinámico sin merge; P2 añade flexibilidad y puede ser superior tras calibrar el número de bloques.

Los resultados entre Linux y Windows son comparables como aceleraciones internas, pero no aíslan el efecto del sistema operativo porque también cambia el hardware y la virtualización. Como trabajo futuro conviene estudiar estabilidad entre repeticiones y realizar una comparación absoluta configuración por configuración con los CSV de ambas plataformas.